

03 · 一条终端消息如何变成模型 API 请求

从键盘输入到 Provider payload，再从流式响应回到终端

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

完整路径有四次关键表示变化：终端输入 -> JavaScript string -> AgentMessage -> pi-ai Message/Context -> Provider 专用 payload。Agent Loop 始终以 AgentMessage 为主表示，只有在 LLM 调用边界才 convertToLlm；只有进入具体 Provider adapter 后才转换成 OpenAI/Anthropic/Google 的请求格式。

1. 选择最小代表场景

场景：Interactive 模式中用户输入“解释 src/main.ts 的入口流程”，按 Enter；当前模型采用 OpenAI Responses API。该路径同时穿过 pi-tui、pi-coding-agent、pi-agent-core、pi-ai，是理解四包边界的最佳主线。

```
Terminal bytes
-> TUI.focusedComponent.handleInput
-> CustomEditor -> EditorState
-> Editor.onSubmit(string)
-> InteractiveMode.getUserInput
-> AgentSession.prompt
-> AgentMessage[]
-> Agent.prompt -> runAgentLoop
-> convertToLlm -> pi-ai Context
-> ModelRuntime.streamSimple
-> Provider.streamSimple
-> provider payload -> SDK/HTTP
```

2. 输入层：TUI 不知道什么是“prompt”

TUI 只把终端数据交给获得焦点的 Component.handleInput。InteractiveMode 把焦点设到 CustomEditor；CustomEditor 先处理 Pi 应用级快捷键，再调用父类 Editor 的通用编辑逻辑。

Editor 保存 lines/cursorLine/cursorCol、撤销栈、粘贴注册表等 UI 状态。提交时 expandPasteMarkers + trim，清空编辑器并触发 onSubmit。到这一点，低层终端事件才第一次变成完整字符串。

3. InteractiveMode 先做“本地路由”

setUpEditorSubmitHandler 不会把所有文本都交给模型。/settings、/model、/login、/compact、/quit 等内置命令在本地执行；以 ! 开头的命令走 Bash；流式期间的普通 Enter 进入 steer 队列。只有普通消息最终通过 getUserInput Promise 被主循环取走并调用 session.prompt。

4. AgentSession.prompt 是产品语义的预处理管线

1. 先识别扩展命令；命中后由扩展自己处理，不进入 Agent。
2. 触发 input 扩展事件；扩展可以 handled 或 transform 文本/图片。
3. 展开 /skill:name 和 Prompt Template。
4. 如果正在 streaming，则根据 steer/followUp 入队，不启动并发 Agent run。
5. 校验当前 model 和认证。
6. 必要时检查上一条 assistant usage 并先做 Compaction。
7. 把展开后的文本与图片构造为 role=user 的 AgentMessage。
8. 触发 before_agent_start，允许注入 custom message 或本轮 system prompt override。
9. 调用 _runAgentPrompt -> agent.prompt。

5. string -> AgentMessage

```
{
  role: "user",
  content: [{ type: "text", text: expandedText }, ...images],
  timestamp: Date.now()
}
```

content 采用 block 数组而非单字符串，使文本、图片、工具结果和后续 provider 转换可以共享结构。Coding Agent 还通过声明合并扩展 AgentMessage，使 bashExecution、custom、branchSummary、compactionSummary 也能进入 transcript。

6. Agent 建立上下文快照，而不是把产品对象传给 Provider

Agent.runPromptMessages 复制 systemPrompt/messages/tools 构造 AgentContext，并组装 AgentLoopConfig (model、reasoning、streamFn、convertToLlm、transformContext、getApiKey、队列与工具 hook)。这样低层 runAgentLoop 只依赖协议对象和函数，不依赖 AgentSession。

7. 真正的 LLM 边界：streamAssistantResponse

10. transformContext: AgentMessage[] -> AgentMessage[]，Coding Agent 用扩展 context 事件实现。
11. convertToLlm: 把 custom/bash/summary 等应用消息转换或过滤为 pi-ai Message[]。
12. 构造 Context={systemPrompt,messages,tools}。
13. 动态解析 API key，支持长任务中 OAuth token 过期后刷新。
14. 调用注入的 streamFunction(model, context, options)。

这是架构上最重要的隔离点：低层 Provider 永远看不到 SessionEntry、Editor、树形会话等产品结构。

8. Coding Agent 的 streamFn 如何进入 pi-ai

createAgentSession 创建 Agent 时注入 streamFn，它不是某个硬编码 Provider，而是调用 ModelRuntime.streamSimple。ModelRuntime.prepareRequest 根据 model.provider 查 Provider、解析 credential/baseUrl/header/env，然后调用 provider.streamSimple。扩展还可在这里经过 before_provider_headers、before_provider_request、after_provider_response。

9. 以 OpenAI Responses 为例的最后一次转换

OpenAI adapter 把统一 Context.messages 转成 ResponseInput: systemPrompt 映射为 system/developer; 用户 TextContent 映射为 input_text; ImageContent 映射 input_image; assistant toolCall 映射 function_call; toolResult 映射 function_call_output。随后 buildParams 加入 model、stream、tools、reasoning、cache 等字段。

真正发出请求的位置是 OpenAI SDK 的 responses.create(params, requestOptions)。因此抓网络 payload 时，最有价值的断点不是 InteractiveMode，而是 provider 的 buildParams/onPayload 和 SDK create。

10. 响应如何反向回到 TUI

```
Provider native events
-> pi-ai AssistantMessageEvent (text_delta/toolcall_delta/...)
-> agent-core message_start/update/end + tool events
-> AgentSessionEvent (persist/extensions/retry)
-> InteractiveMode.handleEvent
-> AssistantMessageComponent / ToolExecutionComponent
-> pi-tui differential render
```

AgentSession 在 message_end 时把 user/assistant/toolResult 持久化；InteractiveMode 订阅同一事件流更新组件。持久化与 UI 都是事件订阅者，而不是 Agent Loop 的硬编码分支。

11. 关键状态流

状态	创建/持有	修改点	生命周期
EditorState	pi-tui Editor	键盘输入/提交	一次编辑到提交后清空
pendingUserInputs	InteractiveMode	onSubmit/getUserInput	等待主循环消费
AgentMessage[]	AgentSession / Agent	prompt、tool result、queue	整个 session transcript
AgentContext snapshot	Agent	每次 run 前复制	一次 Agent run 内可增长
pi-ai Context	streamAssistantResponse	每次 provider request 重建	单次模型请求
Provider payload	具体 adapter	buildParams + onPayload	单次 HTTP/SDK 调用

动态验证方案

实验 1：验证四次表示转换

假设：每层对象都与上一层语义不同

操作：断点设在 Editor.submitValue、AgentSession.prompt 构造 messages、streamAssistantResponse 的 llmContext、OpenAI buildParams。

预期：看到 string -> AgentMessage -> Message/Context -> ResponseCreateParams。

若不一致：若某层缺失，检查是否使用 RPC/Print 或自定义 Provider。

实验 2：验证扩展拦截

假设：input/context/before_provider_request 能在不同边界修改数据

操作：编写小扩展分别给文本加前缀、给 context 注入消息、给 payload 写日志。

预期：三个 hook 依次作用在应用输入、AgentContext、Provider payload。

若不一致：若顺序不同，核对扩展版本和是否命中内置命令。

源码证据索引

文件	关键符号/用途	本报告结论
packages/tui/src/tui.ts + components/editor.ts	handleInput / submitValue	终端数据 -> string
packages/coding-agent/src/modes/interactive/interactive-mode.ts	setupEditorSubmitHandler / getUserInput / run	string -> AgentSession.prompt
packages/coding-agent/src/core/agent-session.ts	prompt / _runAgentPrompt	扩展、模板、认证、AgentMessage
packages/agent/src/agent.ts + agent-loop.ts	runPromptMessages / streamAssistantResponse	AgentMessage -> pi-ai Context
packages/coding-agent/src/core/model-runtime.ts	prepareRequest / streamSimple	Provider 动态分派与认证
packages/ai/src/api/openai-responses.ts	buildParams / client.responses.create	统一 Context -> 真实 OpenAI 请求

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/coding-agent/src/modes/interactive/interactive-mode.ts	只读提交和事件处理相关段落	是
2	packages/coding-agent/src/core/agent-session.ts	精读 prompt()	是
3	packages/agent/src/agent-loop.ts	精读 streamAssistantResponse()	是
4	packages/coding-agent/src/core/messages.ts	理解 AgentMessage -> Message	是
5	packages/ai/src/api/openai-responses.ts	观察最后一公里 Provider 转换	是